

Spring Boot & Java Full Stack

Complete Learning & Interview Preparation Guide

*From Scratch to Associate Software Engineer Level
Covers L1 & L2 Interview Topics*

What you'll learn	
Java Core Refresher	OOP, Collections, Streams, Exceptions
Spring Framework	IoC, DI, Beans, Annotations
Spring Boot	Auto-config, Starters, REST APIs
Data Layer	JPA, Hibernate, JDBC, Transactions
Security	Spring Security, JWT, Auth
Testing	JUnit 5, Mockito, MockMvc
Microservices	Intro, Eureka, Feign, Gateway
Full Stack	Frontend integration, CORS, APIs
Interview Prep	100+ L1 & L2 Q&A

Table of Contents

1. Java Core Refresher (Prerequisites)
2. Build Tools - Maven & Gradle Basics
3. Introduction to Spring Framework
4. Spring Boot - Getting Started
5. Spring Boot Annotations Deep Dive
6. Building REST APIs
7. Exception Handling & Validation
8. Data Access with Spring Data JPA
9. Database Relationships & Transactions
10. Spring Security & JWT
11. Application Properties & Profiles
12. Logging, Actuator & Monitoring
13. Testing Spring Boot Applications
14. Microservices Introduction
15. Caching & Async Processing
16. Full Stack Integration (Frontend + Backend)
17. Best Practices & Design Patterns
18. L1 Interview Questions (50+)
19. L2 Interview Questions (50+)
20. Coding Scenarios & Quick Reference

1. Java Core Refresher (Prerequisites)

Before diving into Spring Boot, you must be comfortable with Java fundamentals. Most L1 interviews start with Java core questions, so this section is critical.

1.1 Object-Oriented Programming (OOP)

Java is an object-oriented language built on four pillars:

- **Encapsulation** - Wrapping data (fields) and methods together; use private fields with public getters/setters.
- **Inheritance** - One class derives from another using *extends*. Promotes code reuse.
- **Polymorphism** - Same method behaves differently. Two types: compile-time (overloading) and runtime (overriding).
- **Abstraction** - Hiding implementation details using *abstract* classes and *interfaces*.

Example: All Four Pillars

```
// Abstraction & Inheritance
abstract class Animal {
    private String name;           // Encapsulation
    public Animal(String name) { this.name = name; }
    public String getName() { return name; }
    public abstract void sound();  // Abstract method
}

class Dog extends Animal {        // Inheritance
    public Dog(String name) { super(name); }
    @Override
    public void sound() {          // Polymorphism (overriding)
        System.out.println(getName() + " says Woof!");
    }
}

public class Main {
    public static void main(String[] args) {
        Animal a = new Dog("Rex"); // Upcasting - runtime polymorphism
        a.sound();                  // Rex says Woof!
    }
}
```

1.2 Class vs Interface vs Abstract Class

Feature	Class	Abstract Class	Interface
Instantiation	Yes	No	No
Methods	Concrete	Both	Abstract + default + static (Java 8+)
Fields	Any	Any	public static final only
Inheritance	Single	Single	Multiple
Keyword	extends	extends	implements

1.3 Collections Framework

The Collections Framework provides ready-to-use data structures. Mastery is mandatory for interviews.

- **List** - Ordered, allows duplicates: ArrayList, LinkedList, Vector.
- **Set** - No duplicates: HashSet, LinkedHashSet, TreeSet.
- **Map** - Key-value pairs: HashMap, LinkedHashMap, TreeMap, ConcurrentHashMap.
- **Queue/Deque** - FIFO/LIFO: PriorityQueue, ArrayDeque.

Common Operations

```
List<String> list = new ArrayList<>();
list.add("Java"); list.add("Spring"); list.add("Boot");

Set<Integer> set = new HashSet<>(Arrays.asList(1, 2, 3, 2)); // {1,2,3}

Map<String, Integer> map = new HashMap<>();
map.put("one", 1); map.put("two", 2);
map.getOrDefault("three", 0); // 0

// Iteration
list.forEach(System.out::println);
map.forEach((k, v) -> System.out.println(k + "=" + v));
```

1.4 Exception Handling

- **Checked exceptions** - Checked at compile time (IOException, SQLException). Must be caught or declared.
- **Unchecked (RuntimeException)** - NullPointerException, ArithmeticException, IllegalArgumentException.
- **Error** - Serious problems (OutOfMemoryError, StackOverflowError). Not usually caught.

```
try {
    int result = 10 / 0;
} catch (ArithmeticException e) {
    System.out.println("Cannot divide by zero: " + e.getMessage());
} catch (Exception e) {
    System.out.println("Generic error");
} finally {
    System.out.println("Always runs");
}

// Try-with-resources (Java 7+) - auto-closes
try (BufferedReader br = new BufferedReader(new FileReader("file.txt"))) {
    System.out.println(br.readLine());
} catch (IOException e) {
    e.printStackTrace();
}
```

1.5 Java 8 Features (Critical for Spring Boot)

Lambda Expressions

```
// Old way
Runnable r1 = new Runnable() {
    public void run() { System.out.println("Hello"); }
};

// Lambda way
Runnable r2 = () -> System.out.println("Hello");

// With parameters
Comparator<Integer> c = (a, b) -> a - b;
```

Functional Interfaces

- **Function<T,R>** - takes T, returns R: $t \rightarrow r$
- **Predicate<T>** - takes T, returns boolean: $t \rightarrow \text{boolean}$
- **Consumer<T>** - takes T, returns void: $t \rightarrow \text{void}$
- **Supplier<T>** - takes nothing, returns T: $() \rightarrow t$

Stream API

```
List<Integer> nums = Arrays.asList(1, 2, 3, 4, 5, 6, 7, 8, 9, 10);

// Filter even, square them, sum
int sum = nums.stream()
    .filter(n -> n % 2 == 0)
    .mapToInt(n -> n * n)
    .sum();    // 220

// Group by
Map<Boolean, List<Integer>> grouped = nums.stream()
    .collect(Collectors.partitioningBy(n -> n % 2 == 0));

// Sorting
List<String> names = Arrays.asList("Bob", "Alice", "Charlie");
List<String> sorted = names.stream()
    .sorted()
    .collect(Collectors.toList());
```

Optional - Avoiding NullPointerException

```
Optional<String> opt = Optional.ofNullable(getName());
String name = opt.orElse("Default");

opt.ifPresent(System.out::println);
String result = opt.map(String::toUpperCase).orElse("EMPTY");
```

2. Build Tools - Maven & Gradle Basics

Build tools manage dependencies, compile code, run tests, and package applications. Maven is most common in Spring Boot projects.

2.1 Maven Fundamentals

Maven uses a **pom.xml** file (Project Object Model) to define project configuration.

Standard Project Structure

```
my-project/
├── pom.xml
├── src/
│   ├── main/
│   │   ├── java/      # Source code
│   │   └── resources/  # Config files, properties
│   └── test/
│       ├── java/      # Test code
│       └── resources/
└── target/            # Compiled output (auto-generated)
```

Sample pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0">
  <modelVersion>4.0.0</modelVersion>
  <groupId>com.example</groupId>
  <artifactId>my-app</artifactId>
  <version>1.0.0</version>
  <packaging>jar</packaging>

  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>3.2.0</version>
  </parent>

  <dependencies>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-web</artifactId>
    </dependency>
  </dependencies>
</project>
```

Important Maven Commands

Command	Purpose
mvn clean	Remove target/ folder
mvn compile	Compile source code
mvn test	Run tests
mvn package	Create JAR/WAR file
mvn install	Install to local Maven repo (~/.m2)
mvn spring-boot:run	Run Spring Boot application
mvn dependency:tree	Show dependency hierarchy

2.2 Maven Lifecycle Phases

Maven follows a defined sequence: **validate** → **compile** → **test** → **package** → **verify** → **install** → **deploy**. Each phase runs the previous ones automatically.

2.3 Gradle (Brief Overview)

Gradle is an alternative build tool that uses Groovy/Kotlin DSL. It is faster and more flexible than Maven, but Maven remains more common in enterprises.

```
// build.gradle example
plugins {
    id 'org.springframework.boot' version '3.2.0'
    id 'java'
}
dependencies {
    implementation 'org.springframework.boot:spring-boot-starter-web'
    testImplementation 'org.springframework.boot:spring-boot-starter-test'
}
```

3. Introduction to Spring Framework

Spring is a powerful, lightweight Java framework that simplifies enterprise application development. Spring Boot is built on top of Spring.

3.1 Why Spring?

- Reduces boilerplate code (no factory patterns, manual wiring).
- Loose coupling through Dependency Injection.
- Easy integration with databases, security, messaging, etc.
- Testable code (POJOs, mocking is easy).
- Huge ecosystem - Spring MVC, Data, Security, Cloud, Batch, etc.

3.2 Inversion of Control (IoC)

IoC means you don't create objects manually - the framework (Spring container) creates and manages them. This **inverts** the traditional control flow.

Traditional vs IoC

```
// Traditional - Tight coupling
class OrderService {
    private PaymentService payment = new PayPalService(); // Hard-coded!
}

// IoC - Loose coupling
class OrderService {
    private PaymentService payment;
    public OrderService(PaymentService payment) { // Injected
        this.payment = payment;
    }
}
```

3.3 Dependency Injection (DI)

DI is the way IoC is implemented - dependencies are *injected* into objects rather than created internally. Three types of injection:

- **Constructor Injection** (recommended) - dependencies passed via constructor.
- **Setter Injection** - dependencies set via setter methods.
- **Field Injection** - dependencies set directly into fields using `@Autowired` (avoid for production).

Example of Each

```
@Service
public class OrderService {
    private final PaymentService payment;

    // 1. Constructor Injection (BEST)
    public OrderService(PaymentService payment) {
        this.payment = payment;
    }
}

@Service
public class OrderService {
```



```

    private PaymentService payment;
    @Autowired // 2. Setter Injection
    public void setPayment(PaymentService payment) {
        this.payment = payment;
    }
}

@Service
public class OrderService {
    @Autowired // 3. Field Injection (avoid)
    private PaymentService payment;
}

```

Note: Constructor injection is preferred because it makes dependencies explicit, supports immutability (*final* fields), and makes unit testing easier.

3.4 Spring Bean

A **bean** is any object managed by the Spring IoC container. Beans are created, wired, and destroyed by Spring.

Bean Scopes

Scope	Description
singleton (default)	One instance per Spring container
prototype	New instance every time it is requested
request	One instance per HTTP request (web only)
session	One instance per HTTP session (web only)
application	One instance per ServletContext (web only)

3.5 Bean Lifecycle

- Container creates instance.
- Dependencies injected.
- `@PostConstruct` method called (custom init logic).
- Bean is ready to use.
- `@PreDestroy` method called before container shutdown.

4. Spring Boot - Getting Started

4.1 What is Spring Boot?

Spring Boot is an opinionated framework on top of Spring that lets you create production-ready applications with minimal configuration. Key features:

- **Auto-configuration** - Configures beans automatically based on classpath.
- **Starter dependencies** - Pre-packaged dependency sets (e.g., spring-boot-starter-web).
- **Embedded servers** - Tomcat, Jetty, or Undertow embedded (no separate deployment).
- **Production-ready** - Built-in metrics, health checks, externalized configuration.
- **No XML** - Java config or annotations.

4.2 Creating a Spring Boot Project

Use **Spring Initializr** (<https://start.spring.io>) to generate a project:

- Choose Maven/Gradle, Java version, Spring Boot version.
- Add dependencies: Web, JPA, MySQL Driver, Lombok, DevTools, etc.
- Download and import into IDE (IntelliJ IDEA, Eclipse, VS Code).

4.3 Project Structure

```
my-springboot-app/
├── pom.xml
├── src/
│   ├── main/
│   │   ├── java/com/example/demo/
│   │   │   ├── DemoApplication.java    # Main class with @SpringBootApplication
│   │   │   ├── controller/            # REST controllers
│   │   │   ├── service/               # Business logic
│   │   │   ├── repository/            # Data access
│   │   │   ├── model/ (or entity/)    # JPA entities
│   │   │   ├── dto/                   # Data Transfer Objects
│   │   │   ├── config/                # Configuration classes
│   │   │   └── resources/
│   │   │       ├── application.properties # Or application.yml
│   │   │       ├── static/             # Static files (HTML, CSS, JS)
│   │   │       └── templates/          # Thymeleaf templates
│   │   └── test/
│   │       ├── java/com/example/demo/
│   │       └── target/
```

4.4 The Main Application Class

```
package com.example.demo;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class DemoApplication {
    public static void main(String[] args) {
        SpringApplication.run(DemoApplication.class, args);
    }
}
```

@SpringBootApplication is a combination of three annotations:

- **@Configuration** - Marks class as a source of bean definitions.
- **@EnableAutoConfiguration** - Triggers Spring Boot's auto-configuration mechanism.
- **@ComponentScan** - Scans the current package and sub-packages for components.

4.5 Your First REST Endpoint

```
@RestController
@RequestMapping("/api")
public class HelloController {

    @GetMapping("/hello")
    public String hello() {
        return "Hello, Spring Boot!";
    }

    @GetMapping("/hello/{name}")
    public String helloName(@PathVariable String name) {
        return "Hello, " + name + "!";
    }
}
```

Run the app and visit *http://localhost:8080/api/hello*

4.6 application.properties vs application.yml

Spring Boot reads configuration from **application.properties** or **application.yml** in `src/main/resources`.

```
# application.properties
server.port=8081
spring.datasource.url=jdbc:mysql://localhost:3306/mydb
spring.datasource.username=root
spring.datasource.password=secret
spring.jpa.hibernate.ddl-auto=update
logging.level.org.springframework=INFO

# application.yml (same config in YAML)
server:
  port: 8081
spring:
  datasource:
    url: jdbc:mysql://localhost:3306/mydb
    username: root
    password: secret
  jpa:
    hibernate:
      ddl-auto: update
```

5. Spring Boot Annotations Deep Dive

5.1 Stereotype Annotations

Annotation	Purpose
@Component	Generic Spring-managed component
@Service	Business logic layer (semantic only)
@Repository	Data access layer; also translates DB exceptions
@Controller	MVC controller returning views
@RestController	@Controller + @ResponseBody (returns JSON/XML)

5.2 Dependency Injection Annotations

- **@Autowired** - Injects dependency by type.
- **@Qualifier("name")** - Resolves ambiguity when multiple beans of same type exist.
- **@Primary** - Marks the default bean to use when multiple exist.
- **@Value("\${property.name}")** - Injects a property value from application.properties.
- **@Resource(name="...")** - JSR-250 alternative to @Autowired + @Qualifier.

```
@Service
public class NotificationService {

    @Value("${app.notification.email:default@example.com}")
    private String adminEmail;

    private final EmailService email;
    private final SmsService sms;

    public NotificationService(@Qualifier("smtpEmail") EmailService email,
                               SmsService sms) {
        this.email = email;
        this.sms = sms;
    }
}
```

5.3 Configuration Annotations

- **@Configuration** - Marks class as bean definition source.
- **@Bean** - Declares a method whose return value is a Spring bean.
- **@ComponentScan** - Tells Spring where to look for components.
- **@PropertySource** - Loads custom properties files.
- **@Profile("dev")** - Activates beans based on profile.

```
@Configuration
public class AppConfig {

    @Bean
    public RestTemplate restTemplate() {
        return new RestTemplate();
    }
}
```

```

@Bean
@Profile("dev")
public DataSource devDataSource() { /* H2 setup */ return null; }

@Bean
@Profile("prod")
public DataSource prodDataSource() { /* MySQL setup */ return null; }
}

```

5.4 Web/REST Annotations

Annotation	Purpose
@RequestMapping	Maps HTTP requests; supports all methods
@GetMapping	Shortcut for @RequestMapping(method=GET)
@PostMapping	For POST requests
@PutMapping	For PUT requests (full update)
@PatchMapping	For PATCH requests (partial update)
@DeleteMapping	For DELETE requests
@PathVariable	Binds URL path variable to method param
@RequestParam	Binds query parameter (?key=val)
@RequestBody	Binds request body (JSON) to object
@RequestHeader	Binds HTTP header to method param
@ResponseStatus	Sets HTTP status code on response
@CrossOrigin	Enables CORS for the endpoint

5.5 Lifecycle Annotations

```

@Component
public class CacheService {

    @PostConstruct
    public void init() {
        System.out.println("Cache initialized after dependencies injected");
    }

    @PreDestroy
    public void cleanup() {
        System.out.println("Cleaning up before bean destruction");
    }
}

```

6. Building REST APIs

6.1 REST Principles

- **Stateless** - Server stores no client context between requests.
- **Resource-based** - Each URL represents a resource (e.g., /users/1).
- **HTTP methods** - GET (read), POST (create), PUT (update), DELETE (remove), PATCH (partial update).
- **Standard status codes** - 200 OK, 201 Created, 400 Bad Request, 401 Unauthorized, 404 Not Found, 500 Internal Server Error.

6.2 Complete CRUD REST Controller

```
@RestController
@RequestMapping("/api/users")
public class UserController {

    private final UserService userService;

    public UserController(UserService userService) {
        this.userService = userService;
    }

    @GetMapping
    public List<User> getAllUsers() {
        return userService.findAll();
    }

    @GetMapping("/{id}")
    public ResponseEntity<User> getUser(@PathVariable Long id) {
        User user = userService.findById(id);
        return ResponseEntity.ok(user);
    }

    @PostMapping
    @ResponseStatus(HttpStatus.CREATED)
    public User createUser(@Valid @RequestBody User user) {
        return userService.save(user);
    }

    @PutMapping("/{id}")
    public User updateUser(@PathVariable Long id,
                          @Valid @RequestBody User user) {
        return userService.update(id, user);
    }

    @DeleteMapping("/{id}")
    @ResponseStatus(HttpStatus.NO_CONTENT)
    public void deleteUser(@PathVariable Long id) {
        userService.delete(id);
    }

    @GetMapping("/search")
    public List<User> search(@RequestParam(required = false) String name,
                           @RequestParam(defaultValue = "0") int page) {
        return userService.search(name, page);
    }
}
```

6.3 ResponseEntity - Full Response Control

Use **ResponseEntity** when you need control over status codes, headers, and body:

```
@GetMapping("/{id}")
public ResponseEntity<User> getUser(@PathVariable Long id) {
    return userService.findById(id)
        .map(user -> ResponseEntity.ok()
            .header("Custom-Header", "value")
            .body(user))
        .orElse(ResponseEntity.notFound().build());
}

// 201 Created with Location header
@PostMapping
public ResponseEntity<User> create(@RequestBody User user) {
    User saved = userService.save(user);
    URI location = ServletUriComponentsBuilder.fromCurrentRequest()
        .path("/{id}")
        .buildAndExpand(saved.getId())
        .toUri();
    return ResponseEntity.created(location).body(saved);
}
```

6.4 DTOs (Data Transfer Objects)

Never expose JPA entities directly! Use DTOs to control what data is exposed and accepted.

```
public class UserDTO {
    private Long id;
    private String name;
    private String email;
    // No password, no internal fields
    // getters & setters
}

@PostMapping
public UserDTO create(@Valid @RequestBody UserDTO dto) {
    User entity = mapToEntity(dto);
    User saved = userService.save(entity);
    return mapToDTO(saved);
}
```

Note: Use libraries like **MapStruct** or **ModelMapper** for automatic DTO-Entity conversion.

7. Exception Handling & Validation

7.1 Why Global Exception Handling?

Without it, exceptions return stack traces to clients - ugly and a security risk. Use `@ControllerAdvice` to handle exceptions centrally.

7.2 Custom Exception

```
public class ResourceNotFoundException extends RuntimeException {
    public ResourceNotFoundException(String message) {
        super(message);
    }
}

// Throw it in your service
@Service
public class UserService {
    public User findById(Long id) {
        return userRepository.findById(id)
            .orElseThrow(() -> new ResourceNotFoundException(
                "User not found with id: " + id));
    }
}
```

7.3 Global Exception Handler

```
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(ResourceNotFoundException.class)
    public ResponseEntity<ErrorResponse> handleNotFound(
        ResourceNotFoundException ex) {
        ErrorResponse error = new ErrorResponse(
            HttpStatus.NOT_FOUND.value(),
            ex.getMessage(),
            LocalDateTime.now()
        );
        return new ResponseEntity<>(error, HttpStatus.NOT_FOUND);
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<Map<String, String>> handleValidation(
        MethodArgumentNotValidException ex) {
        Map<String, String> errors = new HashMap<>();
        ex.getBindingResult().getFieldErrors().forEach(err ->
            errors.put(err.getField(), err.getDefaultMessage()));
        return ResponseEntity.badRequest().body(errors);
    }

    @ExceptionHandler(Exception.class)
    public ResponseEntity<ErrorResponse> handleAll(Exception ex) {
        ErrorResponse error = new ErrorResponse(
            HttpStatus.INTERNAL_SERVER_ERROR.value(),
            "Something went wrong",
            LocalDateTime.now()
        );
        return new ResponseEntity<>(error, HttpStatus.INTERNAL_SERVER_ERROR);
    }
}
```

7.4 Bean Validation (JSR-380)

Add **spring-boot-starter-validation** dependency. Then annotate your DTOs/entities.

```
public class UserDTO {

    @NotNull(message = "ID cannot be null")
    private Long id;

    @NotBlank(message = "Name is required")
    @Size(min = 2, max = 50, message = "Name must be 2-50 characters")
    private String name;

    @Email(message = "Invalid email format")
    @NotBlank
    private String email;

    @Min(value = 18, message = "Age must be at least 18")
    @Max(value = 120)
    private Integer age;

    @Pattern(regexp = "^([0-9]){10}$", message = "Phone must be 10 digits")
    private String phone;

    @Past
    private LocalDate dateOfBirth;
}

@PostMapping
public User create(@Valid @RequestBody UserDTO dto) {
    return userService.save(dto);
}
```

7.5 Common Validation Annotations

Annotation	Purpose
@NotNull	Field must not be null
@NotEmpty	Not null and not empty (collections, strings)
@NotBlank	Not null, not empty, not just whitespace
@Size(min, max)	Validate size of string/collection
@Min / @Max	Numeric range
@Email	Valid email format
@Pattern(regexp)	Match a regex pattern
@Past / @Future	Date in past/future
@Positive / @Negative	Number sign

8. Data Access with Spring Data JPA

8.1 What is JPA?

JPA (Java Persistence API) is a specification for ORM (Object-Relational Mapping). **Hibernate** is the most popular implementation. Spring Data JPA simplifies JPA usage even further.

8.2 Setup

```
<!-- pom.xml -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
<dependency>
  <groupId>com.mysql</groupId>
  <artifactId>mysql-connector-j</artifactId>
  <scope>runtime</scope>
</dependency>

# application.properties
spring.datasource.url=jdbc:mysql://localhost:3306/mydb
spring.datasource.username=root
spring.datasource.password=secret
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true
```

ddl-auto values: *none* (default for MySQL), *validate*, *update*, *create*, *create-drop*. Use **validate** or **none** in production.

8.3 Entity Class

```
@Entity
@Table(name = "users")
public class User {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "full_name", nullable = false, length = 100)
    private String name;

    @Column(unique = true)
    private String email;

    @Enumerated(EnumType.STRING)
    private Role role;

    @Column(name = "created_at", updatable = false)
    @CreationTimestamp
    private LocalDateTime createdAt;

    @UpdateTimestamp
    private LocalDateTime updatedAt;

    // constructors, getters, setters
}

enum Role { USER, ADMIN }
```

8.4 Repository Interface

Spring Data JPA provides auto-implemented repositories - you only define the interface!

```
public interface UserRepository extends JpaRepository<User, Long> {

    // Derived query methods (Spring parses method name)
    Optional<User> findByEmail(String email);
    List<User> findByNameContaining(String name);
    List<User> findByRoleAndCreatedAtAfter(Role role, LocalDateTime date);
    long countByRole(Role role);
    boolean existsByEmail(String email);

    // Custom JPQL
    @Query("SELECT u FROM User u WHERE u.email LIKE %:domain")
    List<User> findByEmailDomain(@Param("domain") String domain);

    // Native SQL
    @Query(value = "SELECT * FROM users WHERE created_at > ?1",
            nativeQuery = true)
    List<User> findRecentUsers(LocalDateTime since);

    // Modifying query
    @Modifying
    @Query("UPDATE User u SET u.role = :role WHERE u.id = :id")
    int updateUserRole(@Param("id") Long id, @Param("role") Role role);
}
```

8.5 Repository Methods You Get for Free

Method	Description
save(entity)	Insert or update
saveAll(entities)	Bulk save
findById(id)	Returns Optional<T>
findAll()	Get all records
findAll(Sort)	Sorted results
findAll(Pageable)	Paginated results
existsById(id)	Check existence
count()	Total count
deleteById(id)	Delete by ID
delete(entity)	Delete entity
deleteAll()	Delete all (carefull!)

8.6 Pagination & Sorting

```
@GetMapping
public Page<User> getUsers(
    @RequestParam(defaultValue = "0") int page,
    @RequestParam(defaultValue = "10") int size,
    @RequestParam(defaultValue = "name") String sortBy) {
    Pageable pageable = PageRequest.of(page, size, Sort.by(sortBy));
    return userRepository.findAll(pageable);
}
```

9. Database Relationships & Transactions

9.1 JPA Relationships

Four types of relationships between entities:

- **@OneToOne** - User ↔ Profile
- **@OneToMany** - Author → Books
- **@ManyToOne** - Book → Author (most common)
- **@ManyToMany** - Students ↔ Courses

@OneToMany / @ManyToOne Example

```
@Entity
public class Author {
    @Id @GeneratedValue
    private Long id;
    private String name;

    @OneToMany(mappedBy = "author",
                cascade = CascadeType.ALL,
                fetch = FetchType.LAZY,
                orphanRemoval = true)
    private List<Book> books = new ArrayList<>();
}
```

```
@Entity
public class Book {
    @Id @GeneratedValue
    private Long id;
    private String title;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "author_id")
    private Author author;
}
```

@ManyToMany Example

```
@Entity
public class Student {
    @Id @GeneratedValue
    private Long id;
    private String name;

    @ManyToMany
    @JoinTable(name = "student_course",
                joinColumns = @JoinColumn(name = "student_id"),
                inverseJoinColumns = @JoinColumn(name = "course_id"))
    private Set<Course> courses = new HashSet<>();
}
```

9.2 Fetch Types: LAZY vs EAGER

- **LAZY** - Related data loaded only when accessed (default for @OneToMany/@ManyToMany). Better performance.
- **EAGER** - Related data loaded immediately (default for @ManyToOne/@OneToOne). Use sparingly.

Note: LAZY fetching can cause **LazyInitializationException** when session is closed. Use **JOIN FETCH** in JPQL or **@EntityGraph** to load related data eagerly when needed.

9.3 Cascade Types

- **PERSIST** - Save parent → save children.
- **MERGE** - Update parent → update children.
- **REMOVE** - Delete parent → delete children.
- **REFRESH** - Refresh parent → refresh children.
- **ALL** - All of the above.

9.4 Transactions with @Transactional

A transaction ensures a group of operations succeed or fail together (ACID properties).

```
@Service
public class TransferService {

    @Transactional
    public void transfer(Long fromId, Long toId, BigDecimal amount) {
        Account from = accountRepo.findById(fromId).orElseThrow();
        Account to = accountRepo.findById(toId).orElseThrow();

        if (from.getBalance().compareTo(amount) < 0) {
            throw new InsufficientFundsException();
        }

        from.setBalance(from.getBalance().subtract(amount));
        to.setBalance(to.getBalance().add(amount));

        accountRepo.save(from);
        accountRepo.save(to);
        // If exception is thrown anywhere, both updates are rolled back
    }
}
```

9.5 Transaction Propagation

Propagation	Behavior
REQUIRED (default)	Join existing tx or create new
REQUIRES_NEW	Always start new tx; suspend existing
SUPPORTS	Use existing tx; run non-tx if none
NOT_SUPPORTED	Run without tx; suspend existing
MANDATORY	Must have existing tx; throw if none
NEVER	Must NOT have tx; throw if exists
NESTED	Nested tx with savepoint

9.6 Transaction Isolation Levels

- **READ_UNCOMMITTED** - Allows dirty reads (lowest isolation).
- **READ_COMMITTED** - No dirty reads (default in most DBs).

- **REPEATABLE_READ** - Same query gives same results within tx.
- **SERIALIZABLE** - Highest isolation; transactions run serially.

10. Spring Security & JWT

10.1 What is Spring Security?

Spring Security handles **authentication** (who are you?) and **authorization** (what can you do?). Add the starter and Spring Boot auto-configures basic security.

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-security</artifactId>
</dependency>
```

10.2 Basic Security Configuration (Spring Boot 3.x)

```
@Configuration
@EnableWebSecurity
public class SecurityConfig {

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        http
            .csrf(csrf -> csrf.disable())
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/api/public/**").permitAll()
                .requestMatchers("/api/admin/**").hasRole("ADMIN")
                .requestMatchers("/api/user/**").hasAnyRole("USER", "ADMIN")
                .anyRequest().authenticated()
            )
            .sessionManagement(sm -> sm
                .sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .httpBasic(Customizer.withDefaults());
        return http.build();
    }

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }
}
```

10.3 JWT Authentication

JWT (JSON Web Token) is the standard for stateless authentication in REST APIs. A JWT has three parts: Header.Payload.Signature

JWT Flow

- User sends username/password to /login.
- Server validates and returns a JWT token.
- Client stores the token (localStorage or cookie).
- Client includes the token in *Authorization: Bearer <token>* header for subsequent requests.
- Server validates the token and processes the request.

Generate & Validate JWT (Simplified)

```
@Component
public class JwtUtil {

    private final String SECRET = "your-256-bit-secret-key-here-min-32-chars";
```

```

private final long EXPIRATION = 24 * 60 * 60 * 1000; // 24 hours

public String generateToken(String username) {
    return Jwts.builder()
        .setSubject(username)
        .setIssuedAt(new Date())
        .setExpiration(new Date(System.currentTimeMillis() + EXPIRATION))
        .signWith(Keys.hmacShaKeyFor(SECRET.getBytes()),
            SignatureAlgorithm.HS256)
        .compact();
}

public String extractUsername(String token) {
    return Jwts.parserBuilder()
        .setSigningKey(Keys.hmacShaKeyFor(SECRET.getBytes()))
        .build().parseClaimsJws(token)
        .getBody().getSubject();
}

public boolean validateToken(String token) {
    try {
        extractUsername(token);
        return true;
    } catch (Exception e) {
        return false;
    }
}
}

```

10.4 Role-Based Authorization

```

@RestController
public class AdminController {

    @PreAuthorize("hasRole('ADMIN')")
    @GetMapping("/admin/users")
    public List<User> getAllUsers() { return userService.findAll(); }

    @PreAuthorize("hasAnyRole('USER','ADMIN')")
    @GetMapping("/profile")
    public User getProfile(Principal principal) {
        return userService.findByUsername(principal.getName());
    }
}

// Enable in config:
// @EnableMethodSecurity(prePostEnabled = true)

```

10.5 Password Encoding

```

@Service
public class AuthService {
    private final PasswordEncoder encoder;
    private final UserRepository userRepo;

    public void register(String username, String password) {
        User user = new User(username, encoder.encode(password)); // Hashed!
        userRepo.save(user);
    }

    public boolean login(String username, String password) {
        User user = userRepo.findByUsername(username).orElseThrow();
        return encoder.matches(password, user.getPassword());
    }
}

```


Note: Never store plain text passwords. Always use BCrypt, Argon2, or similar.

11. Application Properties & Profiles

11.1 Externalized Configuration

Spring Boot loads properties from multiple sources in this priority order (highest first):

- Command-line arguments (`--server.port=9090`)
- Java System properties (`-Dserver.port=9090`)
- OS environment variables
- `application-{profile}.properties` (profile-specific)
- `application.properties` (default)

11.2 Spring Profiles

Profiles let you have different configurations per environment (dev, test, prod).

```
# application.properties (default)
spring.profiles.active=dev

# application-dev.properties
spring.datasource.url=jdbc:h2:mem:devdb
logging.level.root=DEBUG

# application-prod.properties
spring.datasource.url=jdbc:mysql://prod-server/proddb
logging.level.root=WARN
```

Activate a profile via command line: `java -jar app.jar --spring.profiles.active=prod`

11.3 Custom Properties with @ConfigurationProperties

```
# application.yml
app:
  name: My App
  version: 1.0
  admin:
    email: admin@example.com
    phone: 1234567890
  allowed-origins:
    - http://localhost:3000
    - https://myfrontend.com

@Component
@ConfigurationProperties(prefix = "app")
public class AppProperties {
    private String name;
    private String version;
    private Admin admin = new Admin();
    private List<String> allowedOrigins;

    public static class Admin {
        private String email;
        private String phone;
        // getters/setters
    }
    // getters/setters
}

@Service
```

```
public class MyService {  
    private final AppProperties props;  
    public MyService(AppProperties props) { this.props = props; }  
  
    public void doSomething() {  
        System.out.println("App: " + props.getName());  
    }  
}
```

11.4 @ConditionalOn... Annotations

- **@ConditionalOnProperty** - Bean exists only if property is set.
- **@ConditionalOnClass** - Bean exists only if class is on classpath.
- **@ConditionalOnMissingBean** - Bean exists only if no bean of this type exists.
- **@ConditionalOnBean** - Bean exists only if specific bean already exists.

12. Logging, Actuator & Monitoring

12.1 Logging with SLF4J + Logback

Spring Boot uses SLF4J as the facade and Logback as the default implementation.

```
@Service
public class OrderService {
    private static final Logger log = LoggerFactory.getLogger(OrderService.class);

    public void processOrder(Order order) {
        log.debug("Processing order: {}", order.getId());
        try {
            // logic
            log.info("Order {} processed successfully", order.getId());
        } catch (Exception e) {
            log.error("Failed to process order {}", order.getId(), e);
            throw e;
        }
    }
}
```

With **Lombok**, just use `@Slf4j` and skip the `LoggerFactory` line:

```
@Service
@Slf4j
public class OrderService {
    public void processOrder(Order order) {
        log.info("Processing order: {}", order.getId());
    }
}
```

Log Levels (most to least verbose)

- TRACE → DEBUG → INFO → WARN → ERROR
- Set in `application.properties`: `logging.level.com.example=DEBUG`
- Write to file: `logging.file.name=app.log`

12.2 Spring Boot Actuator

Actuator exposes production-ready endpoints to monitor and manage your app.

```
<!-- pom.xml -->
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>

# application.properties
management.endpoints.web.exposure.include=health,info,metrics,env
management.endpoint.health.show-details=always
```

Common Actuator Endpoints

Endpoint	Purpose
/actuator/health	Application health (UP/DOWN)
/actuator/info	Custom app info
/actuator/metrics	JVM, HTTP, DB metrics
/actuator/env	Environment properties

/actuator/beans	All Spring beans
/actuator/mappings	All URL mappings
/actuator/loggers	Loggers (view/change at runtime)
/actuator/threaddump	Thread dump
/actuator/heapdump	Heap dump file

13. Testing Spring Boot Applications

13.1 Testing Pyramid

- **Unit tests** - Test single class in isolation (use Mockito).
- **Integration tests** - Test multiple components together.
- **E2E tests** - Test the whole application (slowest).

13.2 Unit Testing with JUnit 5 & Mockito

```
@ExtendWith(MockitoExtension.class)
class UserServiceTest {

    @Mock
    private UserRepository userRepository;

    @InjectMocks
    private UserService userService;

    @Test
    void shouldReturnUser_whenIdExists() {
        // Arrange
        User user = new User(1L, "Alice");
        when(userRepository.findById(1L)).thenReturn(Optional.of(user));

        // Act
        User result = userService.findById(1L);

        // Assert
        assertNotNull(result);
        assertEquals("Alice", result.getName());
        verify(userRepository, times(1)).findById(1L);
    }

    @Test
    void shouldThrowException_whenIdNotFound() {
        when(userRepository.findById(99L)).thenReturn(Optional.empty());

        assertThrows(ResourceNotFoundException.class,
            () -> userService.findById(99L));
    }
}
```

13.3 Integration Testing with @SpringBootTest

```
@SpringBootTest
@AutoConfigureMockMvc
class UserControllerIntegrationTest {

    @Autowired
    private MockMvc mockMvc;

    @Autowired
    private ObjectMapper objectMapper;

    @MockBean
    private UserService userService;

    @Test
    void shouldReturnUser_whenGetById() throws Exception {
```

```

        User user = new User(1L, "Alice");
        when(userService.findById(1L)).thenReturn(user);

        mockMvc.perform(get("/api/users/1"))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$.id").value(1))
            .andExpect(jsonPath("$.name").value("Alice"));
    }

    @Test
    void shouldCreateUser() throws Exception {
        UserDTO dto = new UserDTO("Bob", "bob@example.com");

        mockMvc.perform(post("/api/users")
            .contentType(MediaType.APPLICATION_JSON)
            .content(objectMapper.writeValueAsString(dto)))
            .andExpect(status().isCreated());
    }
}

```

13.4 Repository Testing with @DataJpaTest

```

@DataJpaTest
class UserRepositoryTest {

    @Autowired
    private TestEntityManager entityManager;

    @Autowired
    private UserRepository userRepository;

    @Test
    void shouldFindByEmail() {
        User user = new User("Alice", "alice@example.com");
        entityManager.persistAndFlush(user);

        Optional<User> found = userRepository.findByEmail("alice@example.com");

        assertTrue(found.isPresent());
        assertEquals("Alice", found.get().getName());
    }
}

```

13.5 Common Testing Annotations

Annotation	Purpose
@SpringBootTest	Loads full application context
@WebMvcTest	Tests only web layer (controllers)
@DataJpaTest	Tests only JPA layer (uses H2)
@MockBean	Replaces a bean with a mock in context
@Mock	Mockito-only mock (no Spring context)
@InjectMocks	Inject mocks into target class
@AutoConfigureMockMvc	Auto-configure MockMvc

14. Microservices Introduction

14.1 What are Microservices?

Microservices is an architectural style where an application is composed of small, independent services that communicate over the network (usually REST or messaging).

Monolith vs Microservices

Aspect	Monolith	Microservices
Deployment	Single unit	Independent services
Scaling	Whole app together	Individual services
Tech stack	Single	Polyglot possible
Database	Shared	DB per service
Complexity	Simple to start	Complex distributed system
Failure	Single point	Isolated

14.2 Spring Cloud Core Components

- **Service Discovery** (Eureka, Consul) - Services register themselves; clients find them by name.
- **API Gateway** (Spring Cloud Gateway, Zuul) - Single entry point; routing, auth, rate limiting.
- **Config Server** - Centralized configuration for all services.
- **Circuit Breaker** (Resilience4j, Hystrix) - Prevents cascading failures.
- **Distributed Tracing** (Sleuth + Zipkin) - Trace requests across services.
- **Load Balancing** (Spring Cloud LoadBalancer) - Client-side load balancing.

14.3 Service Communication

OpenFeign - Declarative REST Client

```
@FeignClient(name = "user-service")
public interface UserClient {

    @GetMapping("/api/users/{id}")
    User getUser(@PathVariable Long id);

    @PostMapping("/api/users")
    User createUser(@RequestBody User user);
}

// Use it in another service
@Service
public class OrderService {
    private final UserClient userClient;
    public Order placeOrder(Long userId) {
        User user = userClient.getUser(userId); // Calls user-service
        // ...
    }
}
```

RestTemplate / WebClient


```

// RestTemplate (synchronous, classic)
@Bean
public RestTemplate restTemplate() { return new RestTemplate(); }

User user = restTemplate.getForObject(
    "http://user-service/api/users/" + id, User.class);

// WebClient (reactive, modern, recommended)
WebClient webClient = WebClient.create();
User user = webClient.get()
    .uri("http://user-service/api/users/{id}", id)
    .retrieve()
    .bodyToMono(User.class)
    .block();

```

14.4 Eureka Service Registry (Brief)

```

// Eureka Server
@SpringBootApplication
@EnableEurekaServer
public class EurekaServerApp { ... }

// Eureka Client (each microservice)
@SpringBootApplication
@EnableEurekaClient // or @EnableDiscoveryClient
public class UserServiceApp { ... }

# application.properties
spring.application.name=user-service
eureka.client.service-url.defaultZone=http://localhost:8761/eureka

```

15. Caching & Async Processing

15.1 Caching with @Cacheable

Caching frequently accessed data improves performance dramatically.

```
// Enable caching in main class
@SpringBootApplication
@EnableCaching
public class App { ... }

@Service
public class ProductService {

    @Cacheable("products")
    public Product findById(Long id) {
        System.out.println("Fetching from DB...");
        return productRepository.findById(id).orElseThrow();
    }

    @CachePut(value = "products", key = "#product.id")
    public Product update(Product product) {
        return productRepository.save(product);
    }

    @CacheEvict(value = "products", key = "#id")
    public void delete(Long id) {
        productRepository.deleteById(id);
    }

    @CacheEvict(value = "products", allEntries = true)
    public void clearAll() {}
}
```

Out of the box, Spring uses a simple in-memory cache. For production, use Redis, Caffeine, or Ehcache.

15.2 Asynchronous Methods with @Async

```
@SpringBootApplication
@EnableAsync
public class App { ... }

@Service
public class EmailService {

    @Async
    public CompletableFuture<Void> sendWelcomeEmail(String email) {
        // Takes 3 seconds, but caller doesn't wait
        Thread.sleep(3000);
        // Send email
        return CompletableFuture.completedFuture(null);
    }
}

// Caller
@Service
public class UserService {
    public void register(User user) {
        userRepo.save(user);
        emailService.sendWelcomeEmail(user.getEmail()); // Non-blocking
        // Returns immediately
    }
}
```

15.3 Scheduled Tasks with @Scheduled

```
@SpringBootApplication
@EnableScheduling
public class App { ... }

@Component
public class ScheduledTasks {

    @Scheduled(fixedRate = 5000) // Every 5 seconds
    public void task1() { ... }

    @Scheduled(fixedDelay = 5000) // 5 sec after previous completes
    public void task2() { ... }

    @Scheduled(cron = "0 0 2 * * *") // Daily at 2 AM
    public void dailyReport() { ... }
}
```

16. Full Stack Integration

16.1 The Big Picture

In a full stack application, Spring Boot serves as the backend REST API while a frontend (React, Angular, or Vue) consumes those APIs.

Typical Architecture



16.2 Enabling CORS

CORS (Cross-Origin Resource Sharing) must be enabled when frontend and backend run on different origins (e.g., :3000 and :8080).

```
// Global CORS configuration
@Configuration
public class CorsConfig {

    @Bean
    public WebMvcConfigurer corsConfigurer() {
        return new WebMvcConfigurer() {
            @Override
            public void addCorsMappings(CorsRegistry registry) {
                registry.addMapping("/api/**")
                    .allowedOrigins("http://localhost:3000",
                                   "https://myapp.com")
                    .allowedMethods("GET", "POST", "PUT", "DELETE")
                    .allowedHeaders("*")
                    .allowCredentials(true);
            }
        };
    }
}
```

Or use `@CrossOrigin` annotation on controllers:

```
@CrossOrigin(origins = "http://localhost:3000")
@RestController
@RequestMapping("/api/users")
public class UserController { ... }
```

16.3 Calling Spring Boot API from React

```
// React component using fetch
import { useEffect, useState } from "react";

function UserList() {
    const [users, setUsers] = useState([]);

    useEffect(() => {
        fetch("http://localhost:8080/api/users", {
            headers: {
                "Authorization": "Bearer " + localStorage.getItem("token")
            }
        })
        .then(response => response.json())
        .then(data => setUsers(data));
    }, []);
}
```

```

    }
  })
  .then(r => r.json())
  .then(data => setUsers(data));
}, []);

return (
  <ul>{users.map(u => <li key={u.id}>{u.name}</li>)}</ul>
);
}

// Using axios (popular HTTP client)
import axios from "axios";

const api = axios.create({
  baseURL: "http://localhost:8080/api",
  headers: { "Content-Type": "application/json" }
});

// Add JWT to every request
api.interceptors.request.use(config => {
  const token = localStorage.getItem("token");
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});

// Use it
const users = await api.get("/users");
const created = await api.post("/users", { name: "Alice" });

```

16.4 Serving Frontend from Spring Boot

For a single-deployment unit, build the frontend and place it in `src/main/resources/static`. Spring Boot will serve `index.html` at the root.

```

# After: npm run build (React)
# Copy build/ contents to src/main/resources/static/
# Now http://localhost:8080/ serves the React app

```

16.5 File Upload Endpoint

```

@RestController
@RequestMapping("/api/files")
public class FileController {

    @PostMapping("/upload")
    public ResponseEntity<String> upload(
        @RequestParam("file") MultipartFile file) throws IOException {
        if (file.isEmpty()) {
            return ResponseEntity.badRequest().body("File is empty");
        }
        Path path = Paths.get("uploads/" + file.getOriginalFilename());
        Files.createDirectories(path.getParent());
        Files.write(path, file.getBytes());
        return ResponseEntity.ok("Uploaded: " + file.getOriginalFilename());
    }

    @GetMapping("/download/{filename}")
    public ResponseEntity<Resource> download(@PathVariable String filename)
        throws MalformedURLException {
        Path path = Paths.get("uploads/" + filename);
        Resource resource = new UrlResource(path.toUri());
        return ResponseEntity.ok()
            .header(HttpHeaders.CONTENT_DISPOSITION,
                "attachment; filename=\"" + filename + "\"");
    }
}

```

```
    }  
    .body(resource);  
}
```

17. Best Practices & Design Patterns

17.1 Layered Architecture

- **Controller** - HTTP entry point. Validates input, calls service, returns response.
- **Service** - Business logic. Coordinates between controller and repository.
- **Repository** - Data access. Uses Spring Data JPA / JdbcTemplate.
- **Entity** - Persistent objects mapped to DB tables.
- **DTO** - Data transfer between layers and to clients.
- **Config** - @Configuration classes for beans and customization.

17.2 Coding Best Practices

- Use **constructor injection** with final fields.
- Never expose entities directly; always use DTOs.
- Validate all input using JSR-380 annotations.
- Centralize exception handling with @RestControllerAdvice.
- Use meaningful HTTP status codes (200, 201, 204, 400, 404, 500).
- Log meaningfully - don't log sensitive data (passwords, tokens).
- Externalize configuration; don't hard-code values.
- Use profiles for environment-specific config.
- Write tests - aim for 70%+ coverage.
- Use proper transaction boundaries.
- Cache when appropriate.
- Document APIs with Swagger/OpenAPI.

17.3 API Documentation with Swagger/OpenAPI

```
<dependency>
  <groupId>org.springdoc</groupId>
  <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
  <version>2.3.0</version>
</dependency>

@RestController
@Tag(name = "Users", description = "User management APIs")
public class UserController {

    @Operation(summary = "Get user by ID")
    @ApiResponses({
        @ApiResponse(responseCode = "200", description = "Found"),
        @ApiResponse(responseCode = "404", description = "Not found")
    })
    @GetMapping("/{id}")
    public User getUser(@Parameter(description = "User ID")
        @PathVariable Long id) {
        return userService.findById(id);
    }
}
```

```
}
```

Access docs at: <http://localhost:8080/swagger-ui.html>

17.4 Common Design Patterns

- **Singleton** - Default Spring bean scope; one instance per container.
- **Factory** - Spring's BeanFactory creates beans on demand.
- **Proxy** - Spring AOP uses proxies for transactions and security.
- **Template Method** - JdbcTemplate, RestTemplate, JmsTemplate.
- **Strategy** - Multiple implementations of an interface (e.g., PaymentMethod).
- **Builder** - Lombok @Builder for fluent object creation.
- **DTO** - Decouple persistence from API contract.

17.5 Lombok - Reducing Boilerplate

```
@Data          // getters, setters, toString, equals, hashCode
@NoArgsConstructor
@AllArgsConstructor
@Builder
@Entity
public class User {
    @Id @GeneratedValue
    private Long id;
    private String name;
    private String email;
}

// Usage:
User user = User.builder()
    .name("Alice")
    .email("alice@example.com")
    .build();
```


18. L1 Interview Questions (50+)

L1 (Level 1) questions typically focus on fundamentals - what things are, basic syntax, and common scenarios. Memorize these definitions and be able to explain them in your own words.

Java Core

Q. What is the difference between JDK, JRE, and JVM?

JVM (Java Virtual Machine) runs Java bytecode. **JRE** (Java Runtime Environment) = JVM + libraries needed to run Java apps. **JDK** (Java Development Kit) = JRE + development tools (javac, javadoc, etc.). You need JDK to develop, JRE to run.

Q. Difference between == and .equals()?

== compares reference (memory address) for objects, value for primitives. **.equals()** compares content/logical equality. Default `Object.equals()` does the same as **==**, but classes like `String` override it for content comparison.

Q. What is the difference between String, StringBuilder, and StringBuffer?

String is immutable. **StringBuilder** is mutable but not thread-safe (faster). **StringBuffer** is mutable and thread-safe (synchronized methods, slower). Use `StringBuilder` unless thread safety is needed.

Q. What are the differences between ArrayList and LinkedList?

ArrayList: backed by dynamic array, $O(1)$ random access, slow inserts/deletes in middle. **LinkedList**: doubly-linked list, $O(1)$ inserts/deletes at ends, $O(n)$ random access. Use `ArrayList` for read-heavy, `LinkedList` for frequent mid-list inserts.

Q. Difference between HashMap and Hashtable?

HashMap is not thread-safe, allows one null key and multiple null values. **Hashtable** is synchronized (thread-safe), doesn't allow null keys/values. `Hashtable` is legacy; use `ConcurrentHashMap` for thread-safe maps.

Q. What is the difference between final, finally, and finalize?

final is a keyword: final variable can't be reassigned, final method can't be overridden, final class can't be extended. **finally** is a block that runs after try/catch regardless of exceptions. **finalize()** was a method called by GC before destroying object (deprecated since Java 9).

Q. Difference between checked and unchecked exceptions?

Checked are checked at compile time (`IOException`, `SQLException`) - must be caught or declared. **Unchecked** (`RuntimeException`) are not checked at compile time (`NullPointerException`, `IllegalArgumentException`).

Q. What is autoboxing and unboxing?

Autoboxing is automatic conversion of primitive to wrapper (`int` → `Integer`). **Unboxing** is the reverse (`Integer` → `int`). Example: `Integer i = 5;` (autobox), `int j = i;` (unbox).

Spring & Spring Boot

Q. What is Spring Boot? Why use it over Spring?

Spring Boot is an opinionated framework that simplifies Spring application development with auto-configuration, starter dependencies, embedded servers, and production-ready features. Compared to Spring, it eliminates XML config, reduces boilerplate, and enables rapid development.

Q. What is @SpringBootApplication?

It's a meta-annotation combining `@Configuration` (bean definitions), `@EnableAutoConfiguration` (auto-configure beans), and `@ComponentScan` (scan for components in current package and subpackages).

Q. What is Inversion of Control (IoC)?

IoC is a design principle where the framework controls object creation and wiring instead of the application code. The Spring IoC container creates, configures, and manages beans.

Q. What is Dependency Injection?

DI is a pattern where dependencies are provided to an object externally rather than created internally. Spring supports constructor, setter, and field injection. Constructor injection is preferred.

Q. Difference between `@Component`, `@Service`, `@Repository`, `@Controller`?

All are stereotype annotations that register a class as a Spring bean. They are functionally similar but semantically different: `@Component` is generic, `@Service` indicates business logic, `@Repository` indicates data access (and translates DB exceptions), `@Controller` indicates web controllers.

Q. Difference between `@RestController` and `@Controller`?

`@RestController` = `@Controller` + `@ResponseBody`. It means every method returns a response body (JSON/XML) directly, not a view name. Use `@Controller` for traditional MVC with views (Thymeleaf, JSP), `@RestController` for REST APIs.

Q. What is `@Autowired`?

It tells Spring to inject a bean of the matching type. Spring resolves by type first, then by name if multiple beans exist (use `@Qualifier`).

Q. Difference between `@RequestParam` and `@PathVariable`?

`@PathVariable` reads from URL path (e.g., `/users/{id}` → `/users/5`). `@RequestParam` reads from query string (e.g., `/users?name=Alice`). Use `@PathVariable` for identifiers, `@RequestParam` for optional filters/parameters.

Q. What is auto-configuration?

Spring Boot scans the classpath and automatically configures beans based on what's present. For example, if H2 is on classpath, it auto-configures an in-memory `DataSource`. Controlled by `@EnableAutoConfiguration`. You can exclude specific configs.

Q. What are Spring Boot starters?

Starters are pre-packaged dependency descriptors. For example, `spring-boot-starter-web` brings Spring MVC, Jackson, Tomcat, and validation in one dependency. Common starters: web, data-jpa, security, test, actuator, validation.

Q. Default bean scope in Spring?

Singleton - one instance per Spring container. Other scopes: prototype, request, session, application, websocket.

Q. What is `application.properties`?

External configuration file (or YAML) where you set properties like server port, database URL, logging levels, etc. Spring Boot reads it automatically from `src/main/resources`.

Q. How do you change the default port?

Set `server.port=9090` in `application.properties`, or pass `--server.port=9090` as a command-line argument.

Q. What is Spring Initializr?

A web-based tool (start.spring.io) that generates a Spring Boot project with chosen dependencies, language, build tool, and Java version.

REST APIs

Q. What is REST?

REST (Representational State Transfer) is an architectural style for web services using HTTP methods (GET, POST, PUT, DELETE) to operate on resources identified by URIs. It is stateless, cacheable, and uses standard status codes.

Q. Difference between PUT and PATCH?

PUT replaces the entire resource (full update). **PATCH** applies partial updates - only the fields you send. PUT is idempotent; PATCH may or may not be.

Q. What is idempotency?

An idempotent operation produces the same result no matter how many times it's called. GET, PUT, DELETE are idempotent; POST is not.

Q. Common HTTP status codes?

200 OK, 201 Created, 204 No Content, 400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 409 Conflict, 500 Internal Server Error.

Q. What is ResponseEntity?

A wrapper class that lets you control the entire HTTP response: status code, headers, and body. Use it when you need flexibility, e.g., `ResponseEntity.status(201).body(user)`.

Q. How do you handle exceptions globally?

Use `@RestControllerAdvice` on a class and define `@ExceptionHandler` methods for each exception type. Spring routes exceptions to these handlers automatically.

Q. What is CORS and how do you enable it?

CORS (Cross-Origin Resource Sharing) controls which origins can access your API. Enable via `@CrossOrigin` annotation on controller, or globally via `WebMvcConfigurer` with `addCorsMappings()`.

Spring Data JPA

Q. What is JPA? What is Hibernate?

JPA (Java Persistence API) is a specification for ORM in Java. Hibernate is the most popular implementation. Spring Data JPA builds on top to remove boilerplate.

Q. What is the difference between save() and saveAndFlush()?

save() persists the entity but may delay the SQL until transaction commits. **saveAndFlush()** immediately synchronizes the change with the database.

Q. What are JpaRepository, CrudRepository, PagingAndSortingRepository?

CrudRepository provides basic CRUD. **PagingAndSortingRepository** extends it with pagination and sorting. **JpaRepository** extends both, adding JPA-specific operations like `flush()`, `saveAndFlush()`, `batch deletes`.

Q. How do you write a custom query?

Use `@Query` with JPQL or native SQL on a repository method. Example: `@Query("SELECT u FROM User u WHERE u.email = :email") User findByEmail(@Param("email") String email);`

Q. What is JPQL?

Java Persistence Query Language - object-oriented version of SQL. Operates on entities and fields, not tables and columns. Database-independent.

Q. Difference between FetchType.LAZY and FetchType.EAGER?

LAZY loads related data only when accessed (default for @OneToMany, @ManyToMany). Better performance. **EAGER** loads everything immediately (default for @ManyToOne, @OneToOne). Can cause N+1 problems.

Q. What is the N+1 query problem?

When loading a parent entity with N children and JPA issues 1 query for the parent and N queries for children. Solve with JOIN FETCH in JPQL, @EntityGraph, or batch fetching.

Testing & Tools

Q. What is JUnit?

Java's most popular unit testing framework. Latest version is JUnit 5 (Jupiter). Uses annotations like @Test, @BeforeEach, @AfterEach, @BeforeAll, @AfterAll.

Q. What is Mockito?

A mocking framework. Lets you create fake objects to isolate the unit under test. Key methods: *mock()*, *when().thenReturn()*, *verify()*.

Q. Difference between @Mock and @MockBean?

@Mock is a Mockito annotation - creates a mock without Spring context. **@MockBean** is Spring's annotation - replaces a real bean in the application context with a mock.

Q. What is Maven?

A build automation and dependency management tool. Uses pom.xml to define project structure, dependencies, plugins, and build lifecycle.

Q. What are common Maven commands?

mvn clean (delete target), *mvn compile* (compile sources), *mvn test* (run tests), *mvn package* (create JAR), *mvn install* (install to local repo), *mvn spring-boot:run* (run Spring Boot app).

19. L2 Interview Questions (50+)

L2 (Level 2) questions go deeper - the *why* and *how* behind concepts. Expect scenario-based and design questions. Be ready with examples from your own projects.

Spring Boot Internals

Q. How does Spring Boot auto-configuration work internally?

Spring Boot uses **@EnableAutoConfiguration** which loads classes listed in `META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports` (or `spring.factories` in older versions). Each auto-configuration class uses **@Conditional** annotations (**@ConditionalOnClass**, **@ConditionalOnMissingBean**, etc.) to decide whether to apply. The order is controlled by **@AutoConfigureBefore**/**@AutoConfigureAfter**.

Q. How would you disable a specific auto-configuration?

Use `@SpringBootApplication(exclude = DataSourceAutoConfiguration.class)` or set `spring.autoconfigure.exclude` in properties.

Q. How does @SpringBootApplication scan components?

It triggers **@ComponentScan**, which scans the package of the annotated class and all sub-packages by default. To scan additional packages, use `@ComponentScan(basePackages = {"com.a", "com.b"})`.

Q. What happens when you call SpringApplication.run()?

It bootstraps the application: creates the `ApplicationContext` (usually `AnnotationConfigServletWebServerApplicationContext` for web apps), registers a `CommandLinePropertySource` for args, refreshes the context (which triggers auto-config, scanning, bean creation), starts the embedded server, runs `CommandLineRunner`/`ApplicationRunner` beans, and returns the context.

Q. How do you customize the embedded Tomcat server?

Implement `WebServerFactoryCustomizer<TomcatServletWebServerFactory>` and override `customize()`. Or set properties like `server.tomcat.max-threads=200`.

Q. Difference between @Bean and @Component?

@Component is applied at class level - Spring scans and registers the class as a bean. **@Bean** is applied at method level in a `@Configuration` class - the method's return value becomes a bean. Use **@Bean** when you don't own the class (third-party), or when you need custom construction logic.

Q. How do you handle circular dependencies?

Best: **redesign** to remove the cycle. If unavoidable: (1) use setter injection instead of constructor, (2) use **@Lazy** on one dependency, or (3) use `ObjectProvider`/`ApplicationContext` lookup. Spring Boot 2.6+ disables circular references by default; enable with `spring.main.allow-circular-references=true`.

DI & Bean Lifecycle

Q. Walk me through the Spring bean lifecycle.

1) Container reads bean definitions. 2) Bean is instantiated. 3) Dependencies injected. 4) `@PostConstruct` / `afterPropertiesSet()` called. 5) Custom init method called. 6) Bean is in use. 7) `@PreDestroy` / `destroy()` called before container shutdown. `BeanPostProcessors` can intercept before/after initialization.

Q. What if two beans have the same type? How to disambiguate?

Use `@Qualifier("beanName")` on the injection point, or mark one bean as `@Primary`. Spring will throw `NoUniqueBeanDefinitionException` otherwise.

Q. Why is constructor injection preferred?

1) Makes dependencies explicit and required. 2) Supports immutability with *final* fields. 3) Easier to unit test (no reflection needed). 4) Detects circular dependencies at startup. 5) Doesn't require Spring to test the class.

Q. Can singleton beans depend on prototype beans? What's the issue?

Yes, but the prototype bean is injected only once - the singleton holds the same instance forever. Fix using **ObjectFactory**, **Provider**, method injection (`@Lookup`), or scoped proxies.

REST & HTTP

Q. How does Spring MVC handle a request internally?

1) Request hits `DispatcherServlet`. 2) `HandlerMapping` finds the matching controller. 3) `HandlerAdapter` invokes the handler method. 4) Return value goes through `HandlerInterceptors` and `HandlerMethodReturnValueHandlers`. 5) `ViewResolver` picks view (or `MessageConverter` for REST). 6) Response is rendered and sent back.

Q. How would you version your REST API?

Common strategies: (1) URI versioning - `/api/v1/users`; (2) header versioning - `X-API-Version: 1`; (3) media-type versioning - `Accept: application/vnd.app.v1+json`; (4) query parameter - `?version=1`. URI versioning is most common for clarity.

Q. How do you implement pagination in a REST API?

Use Spring Data's **Pageable**. The controller accepts a `Pageable` parameter, the repository returns `Page<T>`. Clients request via `?page=0&size=10&sort=name,asc`. The Page response includes content, `totalElements`, `totalPages`, etc.

Q. How would you secure sensitive data in transit and at rest?

In transit: use HTTPS/TLS. **At rest:** encrypt fields (e.g., JPA `AttributeConverter`), encrypt the DB, use a secrets manager (Vault, AWS Secrets Manager) for credentials, never log sensitive data, hash passwords with `BCrypt/Argon2`.

Q. What is content negotiation?

Mechanism where client and server agree on response format using the `Accept` header. Spring supports JSON (default with Jackson), XML, etc. The same endpoint can return JSON or XML based on what the client asks for.

Data & Transactions

Q. How does @Transactional work behind the scenes?

Spring creates a **proxy** around the bean. When a transactional method is called, the proxy starts a transaction (or joins one based on propagation), calls the actual method, then commits or rolls back. Limitations: only works on public methods, fails on self-invocation (calling another `@Transactional` method from the same class).

Q. Why doesn't @Transactional work on self-invocation?

Because Spring's proxy is bypassed when you call `this.someMethod()` from another method of the same class. Fix: extract to a separate bean, or inject self via `ApplicationContext` or `@Lazy` self-reference.

Q. Explain transaction propagation behaviors.

REQUIRED (default) - join existing or create new. **REQUIRES_NEW** - always start new, suspend existing. **SUPPORTS** - join if exists, else non-transactional. **MANDATORY** - must have existing, throw if none. **NEVER** - throw if exists. **NESTED** - nested tx with savepoint.

Q. What is the N+1 query problem? How to solve it?

When fetching N parent entities, JPA executes 1 query for parents and N additional queries (one per parent) for related entities. Solutions: (1) **JOIN FETCH** in JPQL, (2) **@EntityGraph** on repository methods, (3) batch fetching via `hibernate.default_batch_fetch_size`, (4) project to DTOs.

Q. Difference between `findOne()/getReference()` and `findById()`?

findById() hits the DB immediately, returns `Optional`. **getReference()** returns a lazy proxy without hitting DB; useful when you need a reference for a foreign key assignment without loading the entity. Throws `EntityNotFoundException` only on access if it doesn't exist.

Q. What is the difference between `persist()` and `merge()`?

persist() makes a new entity managed and persistent. Throws exception if already managed. **merge()** copies the state of a detached entity onto a managed instance (or creates one); returns the managed instance.

Q. How would you implement soft deletes?

Add a boolean `deleted` column. Use `@SQLDelete` to override DELETE: `@SQLDelete(sql = "UPDATE user SET deleted = true WHERE id = ?")`. Use `@Where` to filter on reads: `@Where(clause = "deleted = false")`.

Q. What is optimistic vs pessimistic locking?

Optimistic: assumes conflicts are rare. Use `@Version` field; throws `OptimisticLockException` on concurrent update. Better performance. **Pessimistic**: locks rows in DB using `SELECT ... FOR UPDATE`. Use `@Lock(LockModeType.PESSIMISTIC_WRITE)`. Use when conflicts are frequent.

Security

Q. How does JWT authentication work?

1) User logs in with credentials. 2) Server validates and generates a signed JWT containing user info and expiration. 3) Client stores token and sends it in Authorization header on every request. 4) Server validates signature and expiration, extracts user info, and processes request. Server is stateless - no session storage needed.

Q. Where should you store JWT on the client?

Trade-offs: **localStorage** - simple but vulnerable to XSS. **HttpOnly cookies** - not accessible to JS (safe from XSS) but vulnerable to CSRF (mitigate with `SameSite=strict` and CSRF tokens). HttpOnly cookies are generally preferred.

Q. What is CSRF and how do you protect against it?

CSRF (Cross-Site Request Forgery) tricks a logged-in user's browser into sending unwanted requests. Spring Security enables CSRF protection by default - requires a token on state-changing requests. For stateless REST APIs with JWT in headers (not cookies), CSRF is often disabled.

Q. How does Spring Security's filter chain work?

Spring Security uses a chain of servlet filters (`SecurityFilterChain`). Each filter handles a concern: authentication, authorization, CSRF, session management, etc. Custom filters can be added at specific positions using `http.addFilterBefore(...)`.

Q. What's the difference between authentication and authorization?

Authentication - verifying who you are (login). **Authorization** - what you can do (roles, permissions). Spring Security uses AuthenticationManager for the first and AccessDecisionManager/AuthorizationManager for the second.

Performance & Scalability

Q. How would you optimize a slow Spring Boot application?

1) **Profile** first (JProfiler, async-profiler, Actuator metrics). 2) Optimize DB: add indexes, fix N+1, use pagination, limit query results. 3) Add caching (Redis, Caffeine). 4) Use async for I/O. 5) Tune JVM (heap size, GC). 6) Connection pool tuning (HikariCP). 7) Reduce response payload. 8) Use HTTP compression.

Q. How does Spring caching work?

Enable with `@EnableCaching`. `@Cacheable` wraps the method in a proxy that checks the cache first - if hit, return cached value; if miss, execute method and cache result. `@CachePut` always updates the cache. `@CacheEvict` removes entries.

Q. How would you handle a memory leak in a Spring Boot app?

1) Take a heap dump (jcmd, jmap, or via Actuator's /heapdump). 2) Analyze with MAT or VisualVM to find dominant objects. 3) Common causes: static collections growing forever, unclosed resources, ThreadLocal not cleared, large session attributes, caches without eviction, listener references not removed.

Q. What is connection pooling? Why is it important?

A connection pool maintains a set of reusable database connections. Opening connections is expensive; reusing them dramatically improves performance. Spring Boot uses **HikariCP** by default. Key config: `spring.datasource.hikari.maximum-pool-size`.

Microservices & Architecture

Q. What are the challenges of microservices?

Distributed system complexity, network failures, eventual consistency, distributed transactions, debugging across services, service discovery, data consistency, deployment complexity, monitoring/tracing, increased operational overhead.

Q. How do microservices communicate?

Synchronous: REST (RestTemplate, WebClient, OpenFeign) or gRPC. **Asynchronous:** Message brokers like RabbitMQ, Kafka, ActiveMQ. Async decouples services and improves resilience.

Q. What is a Circuit Breaker?

A pattern that prevents cascading failures. If a remote call fails repeatedly, the breaker 'opens' and short-circuits future calls (failing fast) for a while. After a timeout, it tries again (half-open). Implementations: Resilience4j (recommended), Hystrix (deprecated).

Q. What is API Gateway?

Single entry point for all client requests. Handles routing, authentication, rate limiting, load balancing, request/response transformation, and observability. Implementations: Spring Cloud Gateway, Zuul, Kong, AWS API Gateway.

Q. What is service discovery?

Mechanism to find network locations of service instances dynamically. Services register with a registry (Eureka, Consul); clients query the registry to find instances. Solves the problem of dynamic IPs in cloud environments.

Q. What is the SAGA pattern?

A pattern for managing distributed transactions across microservices. Instead of one ACID transaction, you use a sequence of local transactions with compensating actions on failure. Two flavors: **Choreography** (services emit events) and **Orchestration** (central coordinator).

Q. What is CQRS?

Command Query Responsibility Segregation - separating write models (commands) from read models (queries), often with different databases. Allows independent scaling and optimization. Often paired with Event Sourcing.

Testing

Q. Difference between @SpringBootTest, @WebMvcTest, @DataJpaTest?

@SpringBootTest - loads full application context (slowest, true integration test). **@WebMvcTest** - loads only web layer (controllers, filters, advice); mock the service layer. **@DataJpaTest** - loads only JPA layer with an in-memory DB.

Q. How do you test exception scenarios?

Use `assertThrows(ExpectedException.class, () -> methodCall())`. With Mockito: `when(repo.findById(any())).thenThrow(new RuntimeException())`.

Q. What is Testcontainers and when do you use it?

Testcontainers spins up real Docker containers for tests (e.g., real MySQL, Redis, Kafka). Use it when in-memory DBs (H2) don't faithfully replicate the production DB, especially for vendor-specific features.

Tricky/Scenario

Q. A REST endpoint is slow. How do you debug it?

1) Add timing logs around suspected sections. 2) Enable SQL logging (`spring.jpa.show-sql=true`) to see DB queries. 3) Use Spring Boot Actuator's metrics. 4) Profile with VisualVM/JProfiler. 5) Look at N+1, missing indexes, network calls. 6) Check thread/connection pool exhaustion. 7) Add distributed tracing (Sleuth/Zipkin/OpenTelemetry).

Q. Your app works locally but fails on the server with a NullPointerException. What do you check?

1) Different profile active (dev vs prod) - missing config. 2) Environment-specific bean not registered. 3) External dependency (DB, API) returning null in prod. 4) Different JDK version. 5) Property values different. 6) Check logs and stack trace carefully.

Q. A user reports they can see another user's data. How do you investigate?

1) Check controller - is the user ID coming from a trusted source (token) or untrusted (path param)? 2) Look for IDOR (Insecure Direct Object Reference) - did you verify the resource belongs to the authenticated user? 3) Check caching - is the cache key user-specific? 4) Check sessions/JWT for leakage. 5) Add authorization checks at service or method level (@PreAuthorize).

Q. Your service must call another service that is sometimes slow. What do you do?

Apply resilience patterns: (1) **Timeout** - never wait forever. (2) **Retry** with exponential backoff (but be careful with non-idempotent calls). (3) **Circuit breaker** to fail fast when the dependency is down. (4) **Bulkhead** to isolate thread pools. (5) **Fallback** response. Resilience4j supports all of these.

Q. How would you design a URL shortener like bit.ly using Spring Boot?

Components: (1) **POST /shorten** - generate short code (base62 of incrementing ID or hash), save mapping. (2) **GET /{code}** - lookup, return 301 redirect. (3) Store in DB (long URL, short code,

created_at, expires_at, click count). (4) Cache hot URLs in Redis. (5) Rate-limit creation. (6) Async-track analytics. (7) Add monitoring.

20. Coding Scenarios & Quick Reference

20.1 Common Coding Tasks

Task 1: Build a complete CRUD API for Books

```
// Entity
@Entity
@Data @NoArgsConstructor @AllArgsConstructor @Builder
public class Book {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    @NotBlank private String title;
    @NotBlank private String author;
    @Min(0) private Double price;
}

// Repository
public interface BookRepository extends JpaRepository<Book, Long> {
    List<Book> findByAuthorContainingIgnoreCase(String author);
}

// Service
@Service
@RequiredArgsConstructor
public class BookService {
    private final BookRepository repo;

    public List<Book> findAll() { return repo.findAll(); }

    public Book findById(Long id) {
        return repo.findById(id).orElseThrow(
            () -> new ResourceNotFoundException("Book " + id));
    }

    public Book create(Book b) { return repo.save(b); }

    public Book update(Long id, Book b) {
        Book existing = findById(id);
        existing.setTitle(b.getTitle());
        existing.setAuthor(b.getAuthor());
        existing.setPrice(b.getPrice());
        return repo.save(existing);
    }

    public void delete(Long id) { repo.deleteById(id); }
}

// Controller
@RestController
@RequestMapping("/api/books")
@RequiredArgsConstructor
public class BookController {
    private final BookService service;

    @GetMapping
    public List<Book> all() { return service.findAll(); }

    @GetMapping("/{id}")
    public Book one(@PathVariable Long id) { return service.findById(id); }

    @PostMapping
    @ResponseStatus(HttpStatus.CREATED)
    public Book create(@Valid @RequestBody Book b) { return service.create(b); }
```

```

    @PutMapping("/{id}")
    public Book update(@PathVariable Long id, @Valid @RequestBody Book b) {
        return service.update(id, b);
    }

    @DeleteMapping("/{id}")
    @ResponseStatus(HttpStatus.NO_CONTENT)
    public void delete(@PathVariable Long id) { service.delete(id); }
}

```

Task 2: Custom Exception + Global Handler

```

public class ResourceNotFoundException extends RuntimeException {
    public ResourceNotFoundException(String msg) { super(msg); }
}

@Data @AllArgsConstructor
class ApiError {
    private int status;
    private String message;
    private LocalDateTime timestamp;
}

@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(ResourceNotFoundException.class)
    public ResponseEntity<ApiError> notFound(ResourceNotFoundException e) {
        return new ResponseEntity<>(
            new ApiError(404, e.getMessage(), LocalDateTime.now()),
            HttpStatus.NOT_FOUND);
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<Map<String,String>> validation(
        MethodArgumentNotValidException e) {
        Map<String,String> errors = new HashMap<>();
        e.getBindingResult().getFieldErrors().forEach(f ->
            errors.put(f.getField(), f.getDefaultMessage()));
        return ResponseEntity.badRequest().body(errors);
    }
}

```

Task 3: Filtering with Pagination

```

@GetMapping("/search")
public Page<Book> search(
    @RequestParam(required = false) String author,
    @RequestParam(defaultValue = "0") int page,
    @RequestParam(defaultValue = "10") int size,
    @RequestParam(defaultValue = "title,asc") String[] sort) {
    Sort.Direction dir = Sort.Direction.fromString(sort[1]);
    Pageable pageable = PageRequest.of(page, size, Sort.by(dir, sort[0]));
    return author != null
        ? repository.findByAuthorContainingIgnoreCase(author, pageable)
        : repository.findAll(pageable);
}

```

20.2 HTTP Status Code Cheat Sheet

Code	Meaning	When to use
200	OK	Successful GET, PUT, PATCH
201	Created	Successful POST creating resource

204	No Content	Successful DELETE; no body
400	Bad Request	Validation failed, malformed JSON
401	Unauthorized	Missing/invalid credentials
403	Forbidden	Authenticated but not allowed
404	Not Found	Resource doesn't exist
409	Conflict	Duplicate, version mismatch
422	Unprocessable	Semantically invalid request
500	Server Error	Unhandled exception
503	Service Unavailable	Maintenance, overloaded

20.3 Annotations Quick Reference

Category	Annotations
Application	@SpringBootApplication, @EnableAutoConfiguration, @ComponentScan
Stereotypes	@Component, @Service, @Repository, @Controller, @RestController
DI	@Autowired, @Qualifier, @Primary, @Value, @Lazy
Configuration	@Configuration, @Bean, @PropertySource, @Profile
Web	@RequestMapping, @GetMapping, @PostMapping, @PutMapping, @DeleteMapping
Web params	@PathVariable, @RequestParam, @RequestBody, @RequestHeader
Validation	@Valid, @NotNull, @NotBlank, @Size, @Email, @Min, @Max
JPA	@Entity, @Table, @Id, @GeneratedValue, @Column, @OneToMany, @ManyToOne
Transactions	@Transactional, @EnableTransactionManagement
Security	@EnableWebSecurity, @PreAuthorize, @PostAuthorize, @Secured
Testing	@SpringBootTest, @WebMvcTest, @DataJpaTest, @MockBean, @Mock, @InjectMocks
Conditional	@ConditionalOnClass, @ConditionalOnProperty, @ConditionalOnMissingBean
Other	@PostConstruct, @PreDestroy, @Async, @Scheduled, @Cacheable, @CacheEvict

20.4 Final Interview Tips

- **Build a project.** A working CRUD app on GitHub trumps memorized definitions. Include README, tests, and proper structure.
- **Know your project deeply.** Be ready to explain every decision: why JPA, why this DB, why this auth strategy.
- **Practice STAR.** Situation, Task, Action, Result - structure your behavioral answers.
- **Speak in trade-offs.** Senior engineers don't say one thing is best - they explain when to use what.
- **Don't bluff.** If you don't know, say so - and explain how you'd find out.
- **Code on paper / whiteboard.** Practice writing controllers, JPA entities, and small algorithms by hand.
- **Master the basics first.** Most candidates fail L1 because they hand-wave fundamentals.

- **Read official docs.** spring.io, baeldung.com - become familiar with them.
- **Mock interviews.** Practice out loud with a friend or yourself in front of a mirror.
- **Be curious.** Ask the interviewer good questions about their tech stack, team, and challenges.

Good luck with your interviews! Remember: consistency beats intensity. Study a little every day, build projects, and the concepts will become second nature. You've got this!